CO2 – AT1 – Database Design and Connectivity

Sample Questions

1. Student Database Design and Connectivity (10 Marks)

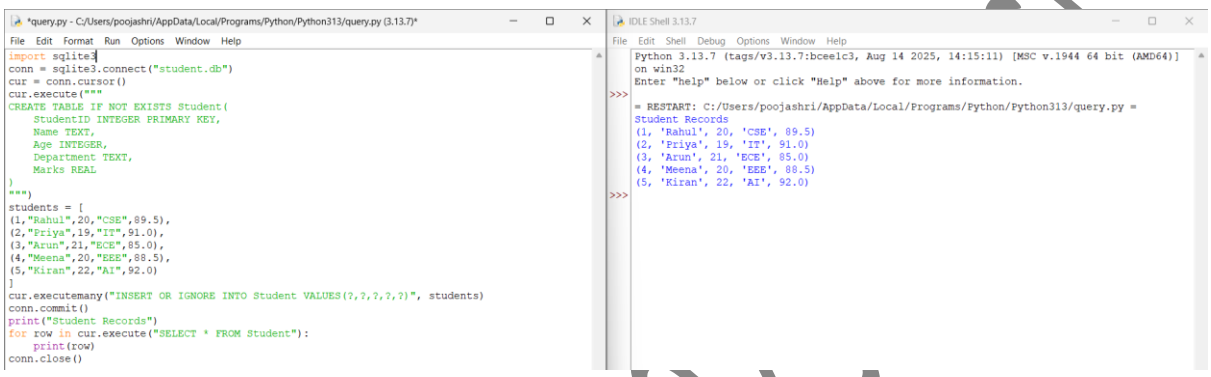
A college wants to automate the management of student records. Design an SQLite database containing a **Student** table with appropriate fields and a primary key. Write a Python program to connect to the SQLite database, create the table if it does not exist, insert at least five student records, and display all the records stored in the table.

CODE:

```
import sqlite3

conn = sqlite3.connect("student.db")

cur = conn.cursor()

cur.execute("""

CREATE TABLE IF NOT EXISTS Student(

   StudentID INTEGER PRIMARY KEY,

   Name TEXT,

   Age INTEGER,

   Department TEXT,

   Marks REAL

)

""")

students = [

(1,"Rahul",20,"CSE",89.5),

(2,"Priya",19,"IT",91.0),

(3,"Arun",21,"ECE",85.0),

(4,"Meena",20,"EEE",88.5),

(5,"Kiran",22,"AI",92.0)

]

cur.executemany("INSERT OR IGNORE INTO Student VALUES(?,?,?,?,?)", students)

conn.commit()
```

```
print("Student Records")
for row in cur.execute("SELECT * FROM Student"):
    print(row)
conn.close()
```

OUTPUT:



2. Library Management Database (10 Marks)

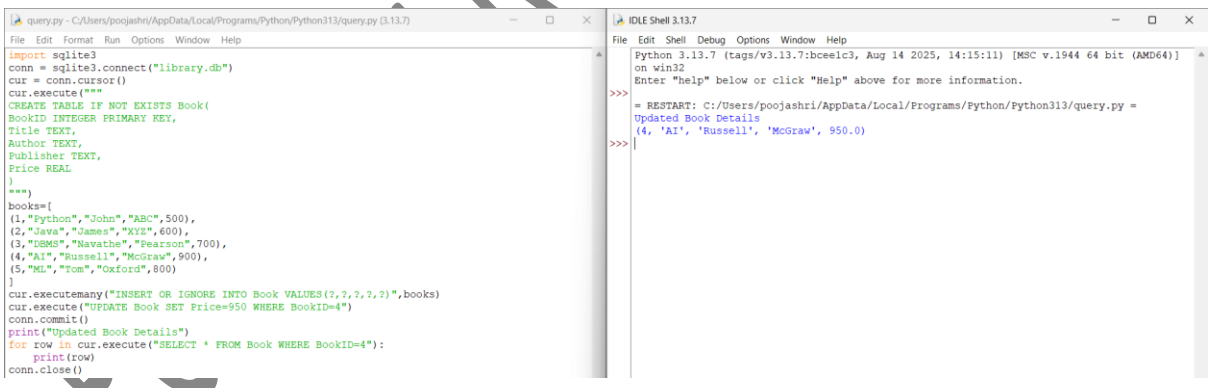
A library plans to maintain information about its books using an SQLite database. Design a **Book** table containing suitable attributes such as Book ID, Title, Author, Publisher, and Price. Write a Python program to establish a connection with the database, create the table, insert sample records, update the price of a selected book, and retrieve the updated information.

CODE:

```
import sqlite3
conn = sqlite3.connect("library.db")
cur = conn.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Book(
BookID INTEGER PRIMARY KEY,
Title TEXT,
Author TEXT,
Publisher TEXT,
Price REAL
)
```

```
""")
books=[
(1,"Python","John","ABC",500),
(2,"Java","James","XYZ",600),
(3,"DBMS","Navathe","Pearson",700),
(4,"AI","Russell","McGraw",900),
(5,"ML","Tom","Oxford",800)
]
cur.executemany("INSERT OR IGNORE INTO Book VALUES(?,?,?,?,?)",books)
cur.execute("UPDATE Book SET Price=950 WHERE BookID=4")
conn.commit()
print("Updated Book Details")
for row in cur.execute("SELECT * FROM Book WHERE BookID=4"):
    print(row)
conn.close()
```

OUTPUT:



3. Employee and Department Database Design (10 Marks)

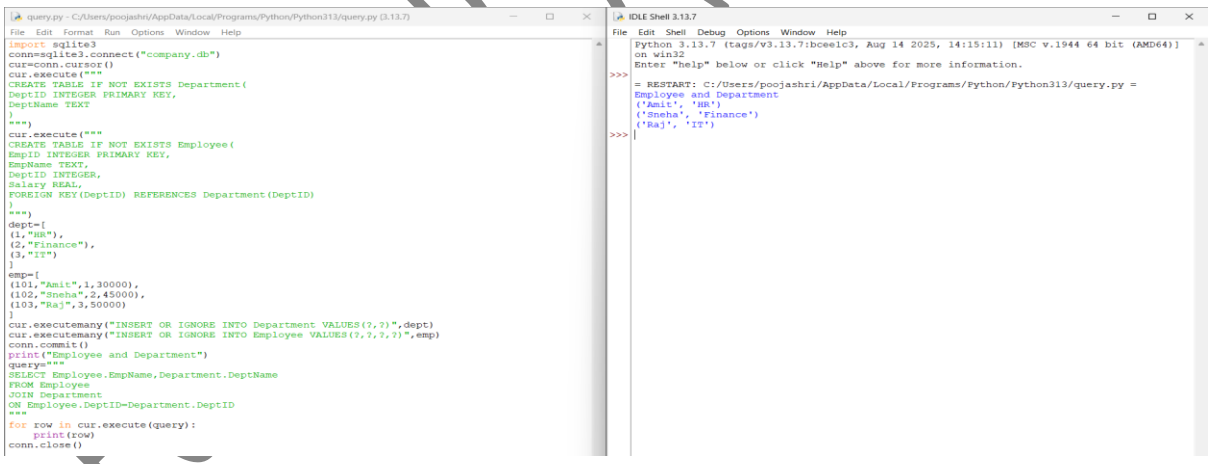
A company wants to store employee information and department details in an SQLite database. Design two related tables, **Department** and **Employee**, by identifying appropriate primary and foreign keys. Write a Python program to connect to the database, create both tables, insert sample data into each table, and display the employee name along with the corresponding department name using an SQL JOIN query.

CODE:

```
import sqlite3
conn=sqlite3.connect("company.db")
cur=conn.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Department(
DeptID INTEGER PRIMARY KEY,
DeptName TEXT
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS Employee(
EmpID INTEGER PRIMARY KEY,
EmpName TEXT,
DeptID INTEGER,
Salary REAL,
FOREIGN KEY(DeptID) REFERENCES Department(DeptID)
)
""")
dept=[
(1,"HR"),
(2,"Finance"),
(3,"IT")
]
emp=[
(101,"Amit",1,30000),
(102,"Sneha",2,45000),
(103,"Raj",3,50000)
]
```

```
cur.executemany("INSERT OR IGNORE INTO Department VALUES(?,?)",dept)
cur.executemany("INSERT OR IGNORE INTO Employee VALUES(?,?,?,?)",emp)
conn.commit()
print("Employee and Department")
query="""
SELECT Employee.EmpName,Department.DeptName
FROM Employee
JOIN Department
ON Employee.DeptID=Department.DeptID
"""
for row in cur.execute(query):
    print(row)
conn.close()
```

OUTPUT:



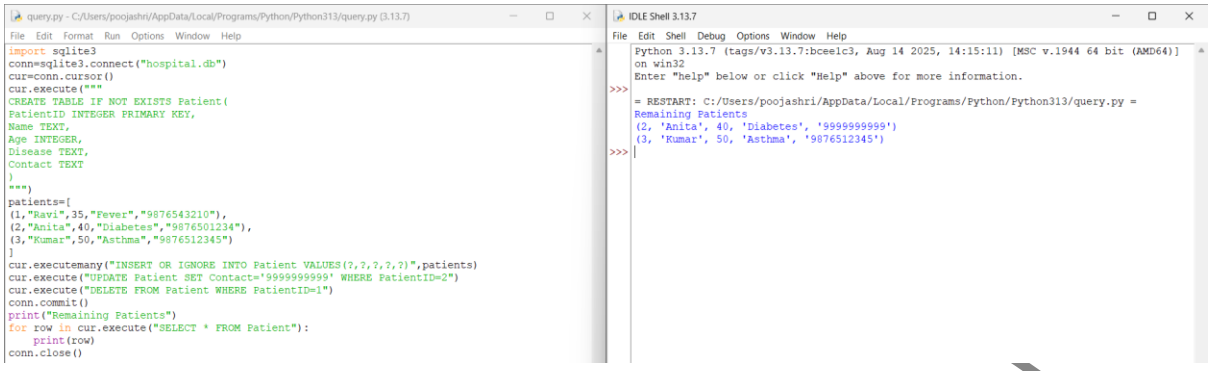
4. Hospital Patient Database (10 Marks)

A hospital requires a database to manage patient information. Design an SQLite database with a **Patient** table containing suitable fields and constraints. Write a Python program to connect to the database, create the table, insert patient records, update the contact number of a specified patient, delete a discharged patient's record, and display the remaining records.

CODE:

```
import sqlite3
conn=sqlite3.connect("hospital.db")
cur=conn.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Patient(
PatientID INTEGER PRIMARY KEY,
Name TEXT,
Age INTEGER,
Disease TEXT,
Contact TEXT
)
""")
patients=[
(1,"Ravi",35,"Fever","9876543210"),
(2,"Anita",40,"Diabetes","9876501234"),
(3,"Kumar",50,"Asthma","9876512345")
]
cur.executemany("INSERT OR IGNORE INTO Patient VALUES(?,?,?,?,?)",patients)
cur.execute("UPDATE Patient SET Contact='9999999999' WHERE PatientID=2")
cur.execute("DELETE FROM Patient WHERE PatientID=1")
conn.commit()
print("Remaining Patients")
for row in cur.execute("SELECT * FROM Patient"):
    print(row)
conn.close()
```

OUTPUT:



5. Online Course Registration System (10 Marks)

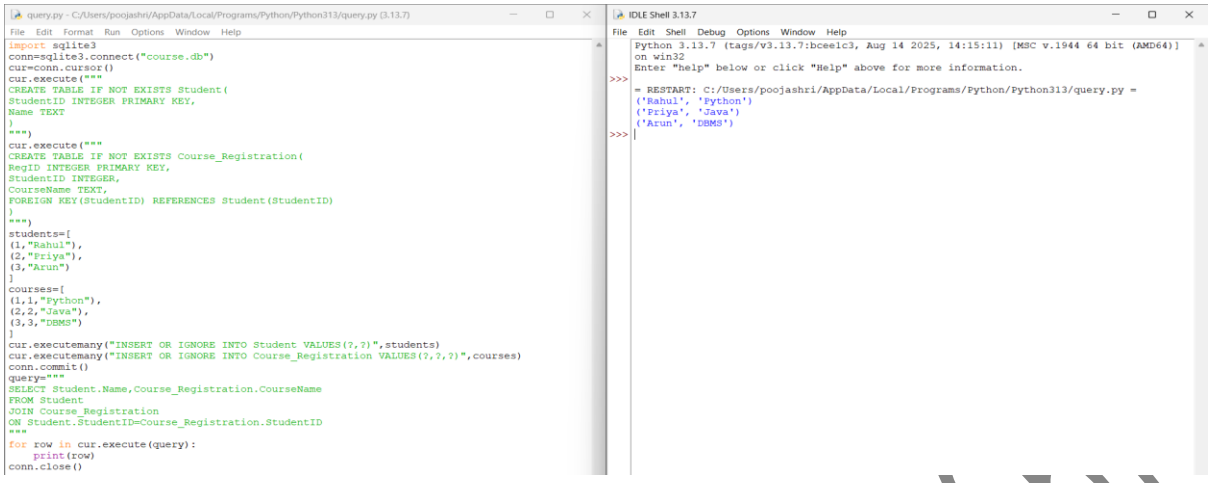
An online learning platform needs to maintain information about students and the courses they have registered for. Design two related SQLite tables, **Student** and **Course_Registration**, using appropriate primary and foreign keys. Write a Python program to connect to the database, create the tables, insert sample records, and retrieve the names of students along with the courses they have registered for using an SQL JOIN operation.

CODE:

```
import sqlite3

conn=sqlite3.connect("course.db")

cur=conn.cursor()

cur.execute("""

CREATE TABLE IF NOT EXISTS Student(

StudentID INTEGER PRIMARY KEY,

Name TEXT

)

""")

cur.execute("""

CREATE TABLE IF NOT EXISTS Course_Registration(

RegID INTEGER PRIMARY KEY,

StudentID INTEGER,

CourseName TEXT,

FOREIGN KEY(StudentID) REFERENCES Student(StudentID)

)
```

```
""")
students=[
(1,"Rahul"),
(2,"Priya"),
(3,"Arun")
]
courses=[
(1,1,"Python"),
(2,2,"Java"),
(3,3,"DBMS")
]
cur.executemany("INSERT OR IGNORE INTO Student VALUES(?,?)",students)
cur.executemany("INSERT OR IGNORE INTO Course_Registration VALUES(?,?,?)",courses)
conn.commit()
query="""
SELECT Student.Name,Course_Registration.CourseName
FROM Student
JOIN Course_Registration
ON Student.StudentID=Course_Registration.StudentID
"""
for row in cur.execute(query):
    print(row)
conn.close()
```

OUTPUT:



6. Banking Account Management System (10 Marks)

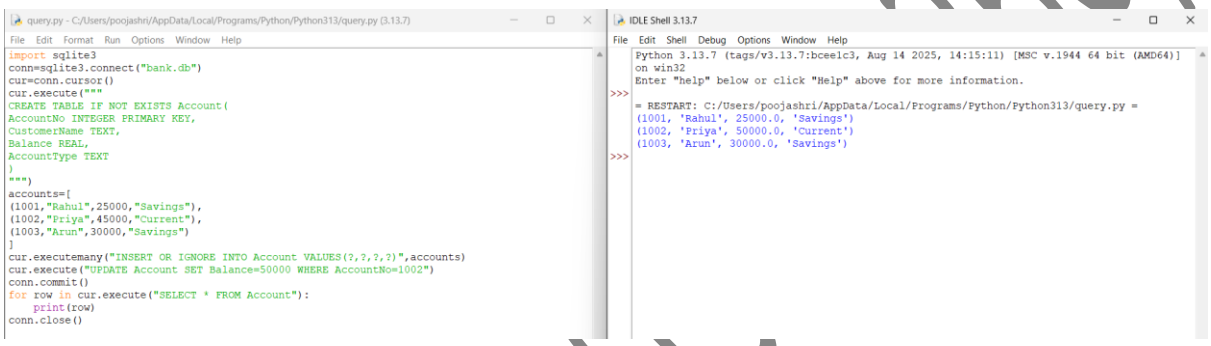
A bank wants to maintain customer account information using an SQLite database. Design an appropriate database containing an **Account** table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert customer account details, update the account balance of a specified customer, and display all customer records.

CODE:

```
import sqlite3

conn=sqlite3.connect("bank.db")

cur=conn.cursor()

cur.execute("""

CREATE TABLE IF NOT EXISTS Account(

AccountNo INTEGER PRIMARY KEY,

CustomerName TEXT,

Balance REAL,

AccountType TEXT

)

""")

accounts=[

(1001,"Rahul",25000,"Savings"),

(1002,"Priya",45000,"Current"),

(1003,"Arun",30000,"Savings")
```

```
]
cur.executemany("INSERT OR IGNORE INTO Account VALUES(?,?,?,?)",accounts)
cur.execute("UPDATE Account SET Balance=50000 WHERE AccountNo=1002")
conn.commit()
for row in cur.execute("SELECT * FROM Account"):
    print(row)
conn.close()
```

OUTPUT:



7. Inventory Management System (10 Marks)

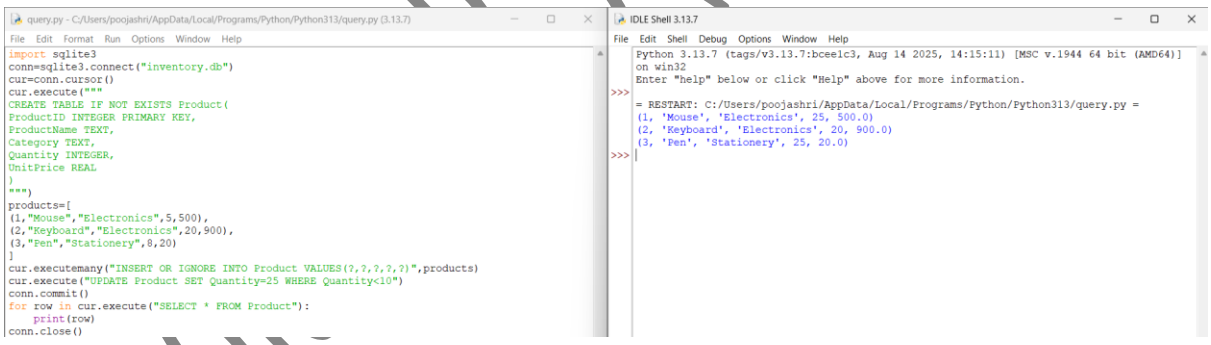
A retail store needs to maintain product inventory using SQLite. Design a **Product** table containing Product ID, Product Name, Category, Quantity, and Unit Price. Write a Python program to create the database, insert sample products, identify products with quantity less than 10, update their stock quantity, and display the updated inventory.

CODE:

```
import sqlite3
conn=sqlite3.connect("inventory.db")
cur=conn.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Product(
ProductID INTEGER PRIMARY KEY,
ProductName TEXT,
Category TEXT,
Quantity INTEGER,
```

```
UnitPrice REAL
)
""")
products=[
(1,"Mouse","Electronics",5,500),
(2,"Keyboard","Electronics",20,900),
(3,"Pen","Stationery",8,20)
]
cur.executemany("INSERT OR IGNORE INTO Product VALUES(?,?,?,?,?)",products)
cur.execute("UPDATE Product SET Quantity=25 WHERE Quantity<10")
conn.commit()
for row in cur.execute("SELECT * FROM Product"):
    print(row)
conn.close()
```

OUTPUT:



8. Course and Faculty Management System (10 Marks)

A university wants to maintain details of faculty members and the courses they teach. Design two related tables, **Faculty** and **Course**, using appropriate primary and foreign keys. Write a Python program to create the database, insert sample records, and display each faculty member along with the courses assigned using an SQL JOIN query.

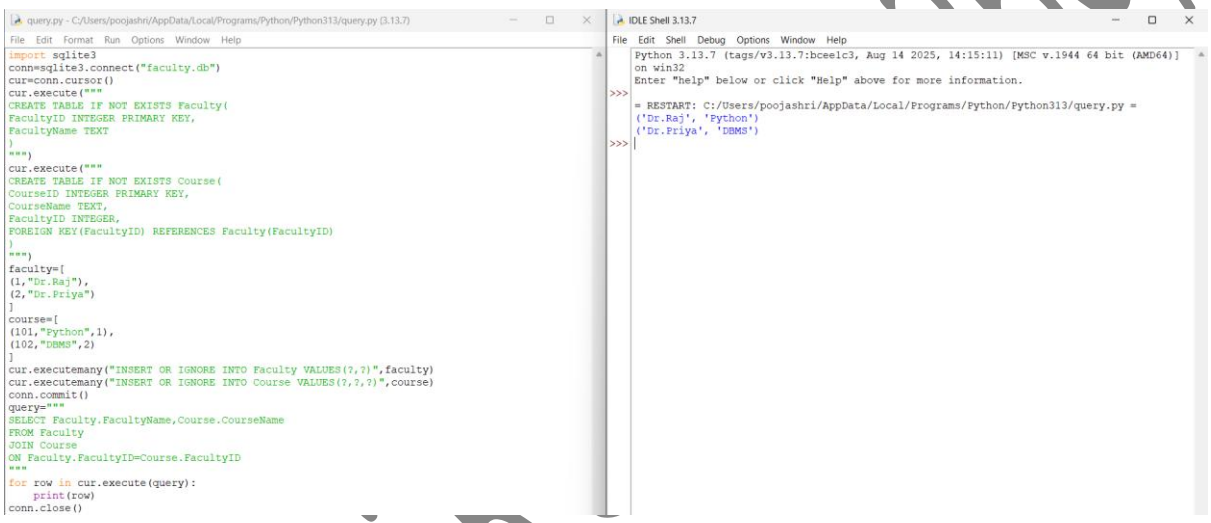
CODE:

```
import sqlite3
conn=sqlite3.connect("faculty.db")
```

```
cur=conn.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Faculty(
FacultyID INTEGER PRIMARY KEY,
FacultyName TEXT
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS Course(
CourseID INTEGER PRIMARY KEY,
CourseName TEXT,
FacultyID INTEGER,
FOREIGN KEY(FacultyID) REFERENCES Faculty(FacultyID)
)
""")
faculty=[
(1,"Dr.Raj"),
(2,"Dr.Priya")
]
course=[
(101,"Python",1),
(102,"DBMS",2)
]
cur.executemany("INSERT OR IGNORE INTO Faculty VALUES(?,?)",faculty)
cur.executemany("INSERT OR IGNORE INTO Course VALUES(?,?,?)",course)
conn.commit()
query="""
SELECT Faculty.FacultyName,Course.CourseName
```

```
FROM Faculty

JOIN Course

ON Faculty.FacultyID=Course.FacultyID

"""

for row in cur.execute(query):

    print(row)

conn.close()
```

OUTPUT:



9. Vehicle Registration Database (10 Marks)

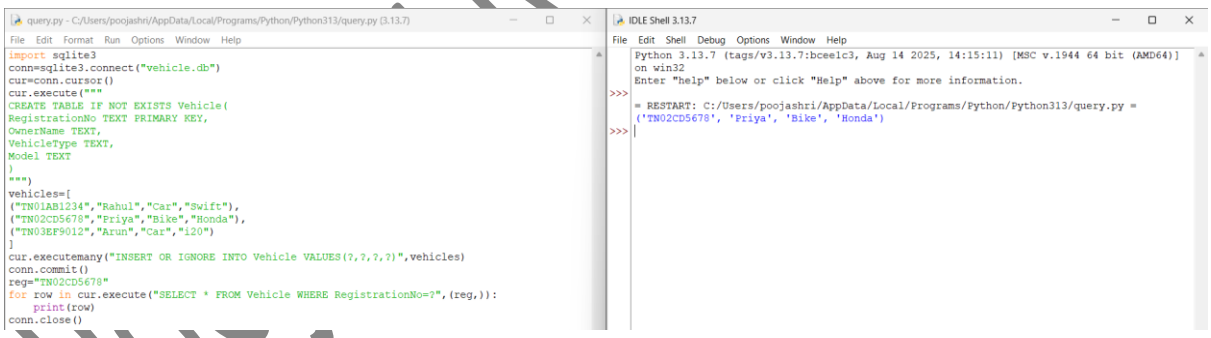
A transport department plans to maintain vehicle registration details in an SQLite database. Design a **Vehicle** table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert vehicle records, search for a vehicle using its registration number, and display the retrieved information.

CODE:

```
import sqlite3

conn=sqlite3.connect("vehicle.db")

cur=conn.cursor()

cur.execute("""

CREATE TABLE IF NOT EXISTS Vehicle(

RegistrationNo TEXT PRIMARY KEY,

OwnerName TEXT,
```

```
VehicleType TEXT,
Model TEXT
)
""")
vehicles=[
("TN01AB1234","Rahul","Car","Swift"),
("TN02CD5678","Priya","Bike","Honda"),
("TN03EF9012","Arun","Car","i20")
]
cur.executemany("INSERT OR IGNORE INTO Vehicle VALUES(?,?,?,?)",vehicles)
conn.commit()
reg="TN02CD5678"
for row in cur.execute("SELECT * FROM Vehicle WHERE RegistrationNo=?",(reg,)):
    print(row)
conn.close()
```

OUTPUT:



10. Hospital Appointment Management System (10 Marks)

A hospital maintains separate tables for **Doctors** and **Appointments**. Design both tables using appropriate primary and foreign keys. Write a Python program to create the tables, insert sample records, and display the doctor name along with the appointment details using a JOIN query.

CODE:

```
import sqlite3
conn=sqlite3.connect("appointment.db")
```

```
cur=conn.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Doctor(
DoctorID INTEGER PRIMARY KEY,
DoctorName TEXT,
Specialization TEXT
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS Appointment(
AppointmentID INTEGER PRIMARY KEY,
DoctorID INTEGER,
PatientName TEXT,
Date TEXT,
FOREIGN KEY(DoctorID) REFERENCES Doctor(DoctorID)
)
""")
doctors=[
(1,"Dr.Ravi","Cardiology"),
(2,"Dr.Anita","Neurology")
]
appointments=[
(101,1,"Rahul","2026-08-03"),
(102,2,"Priya","2026-08-04")
]
cur.executemany("INSERT OR IGNORE INTO Doctor VALUES(?,?,?)",doctors)
cur.executemany("INSERT OR IGNORE INTO Appointment VALUES(?,?,?,?)",appointments)
conn.commit()
```

```
query="""
SELECT Doctor.DoctorName,
Appointment.PatientName,
Appointment.Date
FROM Doctor
JOIN Appointment
ON Doctor.DoctorID=Appointment.DoctorID
"""
for row in cur.execute(query):
    print(row)
conn.close()
```

OUTPUT: